

CTiC
Certification Together International Conference
Organized by AKKODIS

SONNX

Enabling Certifiable Machine Learning through Unambiguous ONNX Semantics

Eric Jenn (IRT Saint Exupery, WG Co-lead)
eric.jenn@irt-saintexupery.com

Overview

- **Who**: the working group…
- **Where**: the context…
- **Why**: the objectives and the issues…
- **How**: the approach…
- **What**: Some first results…
- **Status** and next steps…



Who.

The SONNX working group

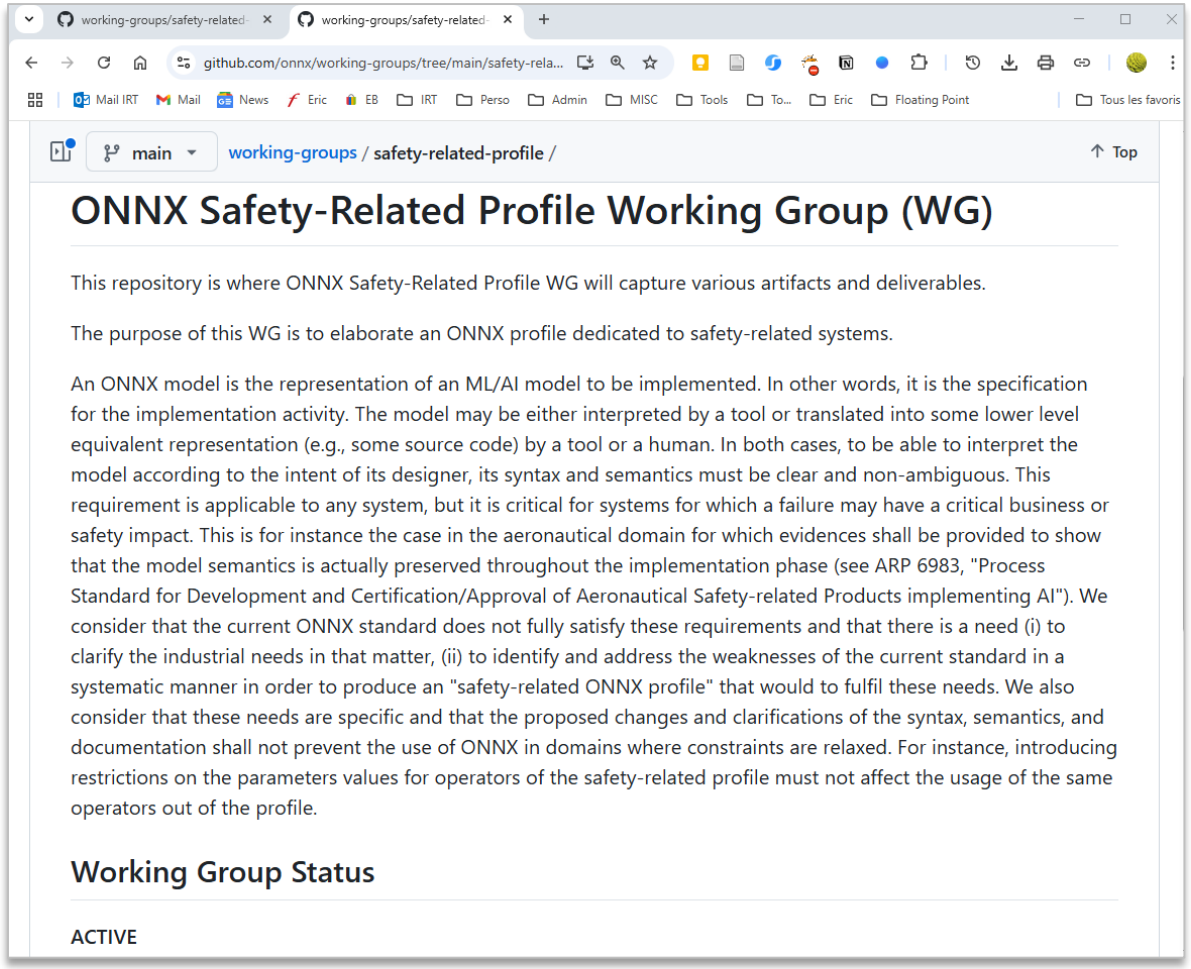
Overview

- Started in 2024
- Hosted by the ONNX community
- Core team of industrial and academic participants (around 10p)

We Need You!



- Around 30 bi-weekly meetings
- Multiple working sessions on formal specification, reviews, etc.



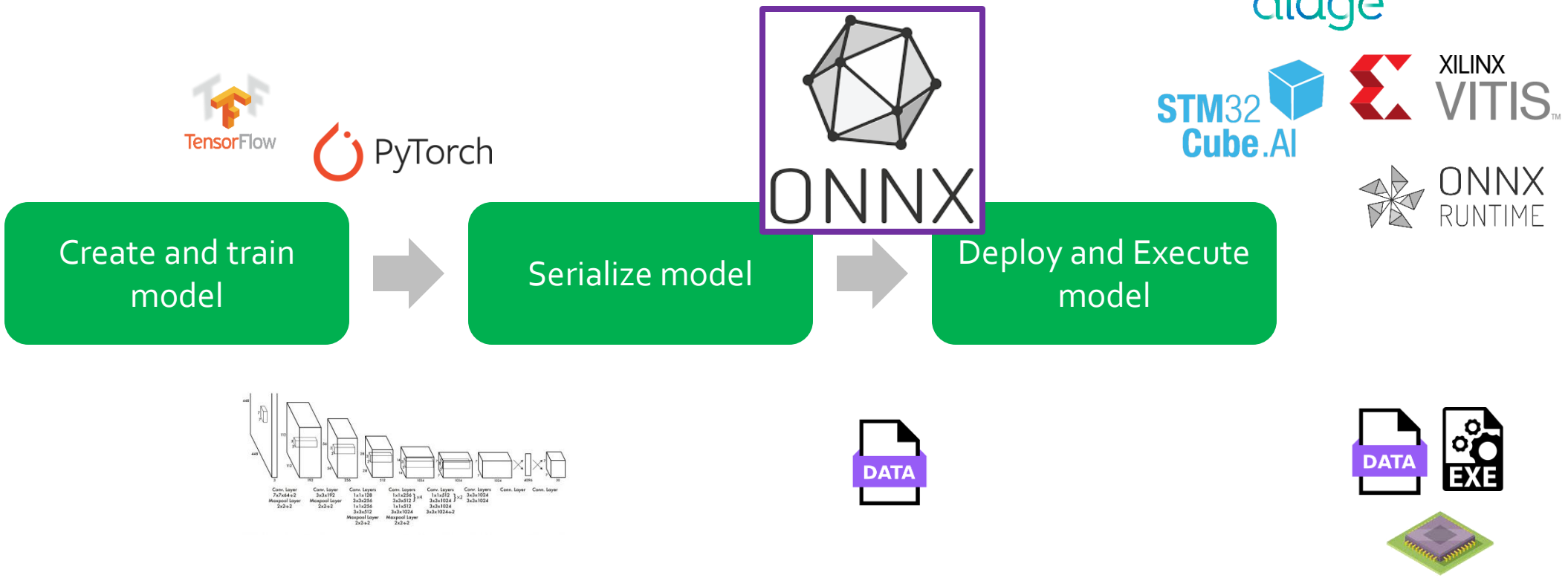
(link to the repo at the end of the presentation)



Where.

Context

A Typical ML workflow



Context
ONNX

- A set of operators and a graph execution model
- An API
- An Intermediate Representation (IR) described using Protobuf

- A "reference implementation" coded in Python
- A runtime (ONNXruntime) [managed as a separate project in ONNX Runtime | Home]

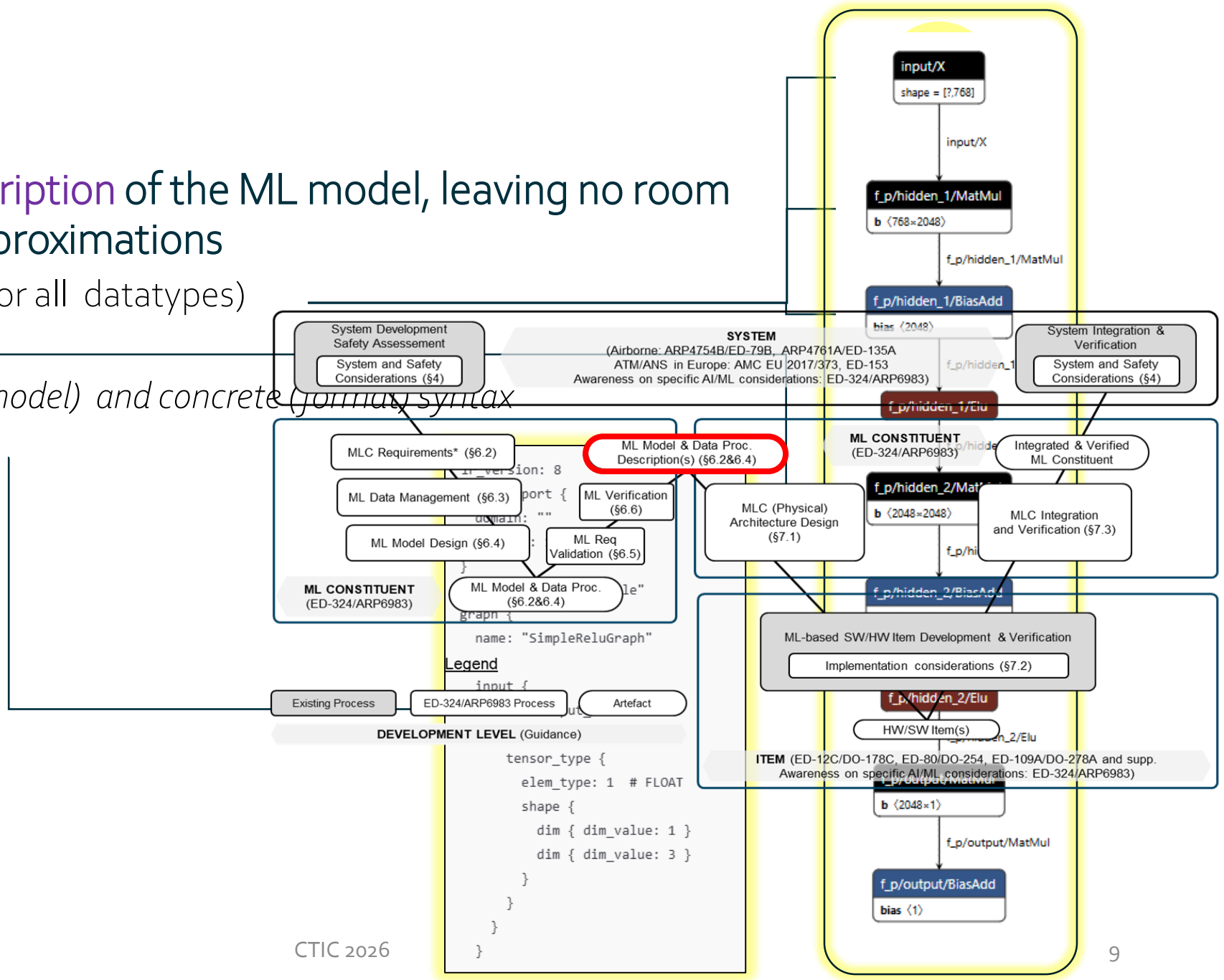




Why.

Objectives

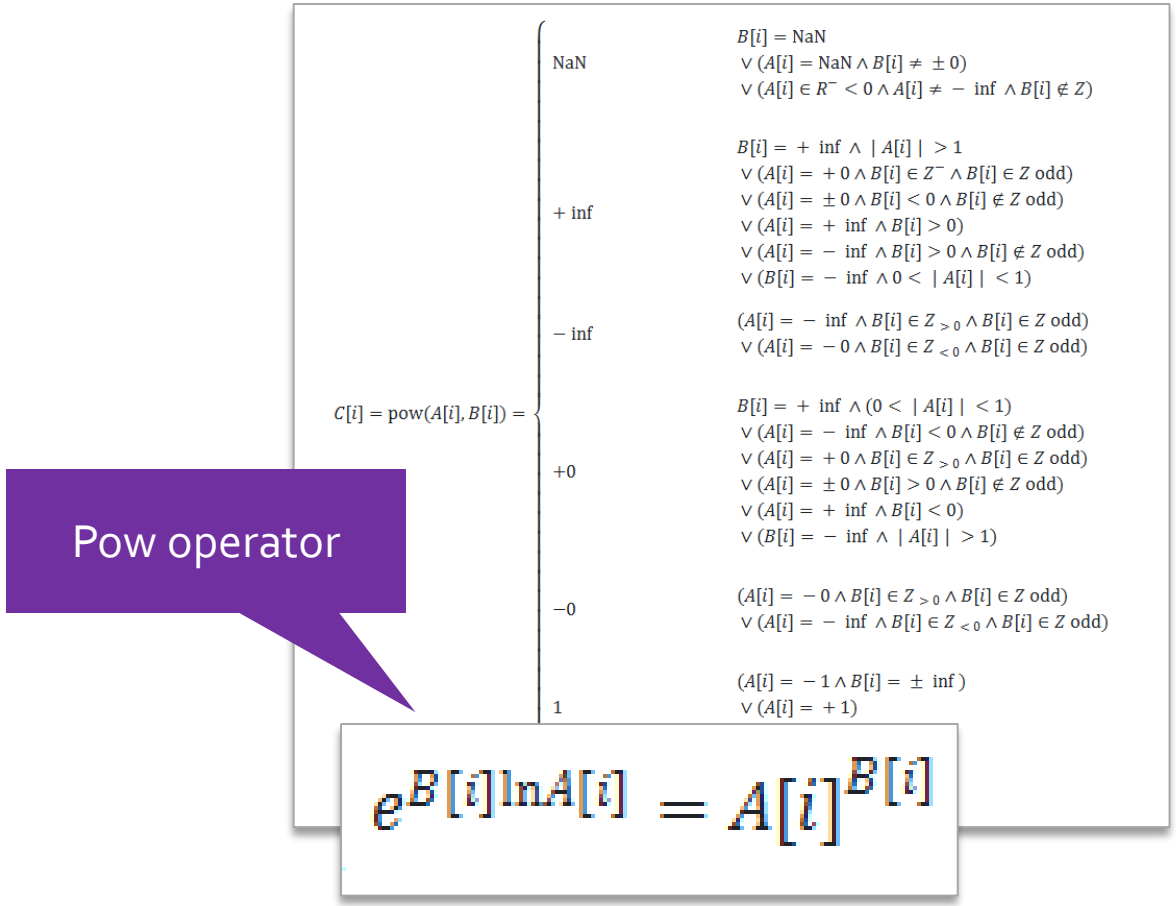
- Provide an accurate description of the ML model, leaving no room to interpretation and approximations
  - The operator semantics (for all datatypes)
  - The graph semantics
  - *The ONNX abstract (metamodel) and concrete (formal) syntax*



Do we really understand what we are using?

Data types, null tensors,…

- Abstraction
  - Abstraction is generally good.
    Is it?
- Non ML-specific
  - What is exactly an integer addition?
  - What is exactly a floating point number?
    - What are the IEEE special values?
- ML-specific (example)
  - What is a null tensor?
  - How does an operator behave when the tensor is null or is a scalar?
  - Can the values in a null tensor "resurrect"?



Facts: Empirical evidences of problems

ONNX failed conversion survey

- Converters fail…
- … often with the bad mode…
- (… but no root cause leading to the spec…)

> **Finding 4.** Location: Most failures are in *Node Conversion* (74%).
> **Finding 5.** Symptom: The most common symptoms in DL model converters are *Crash* (56%) and *Wrong Model* (33%).
> **Finding 6.** Causes: *Crashes* are largely due to *Incompatibilities* and *Type Problems*. *Wrong models* are largely due to *Type Problems* and *Algorithmic Errors*.

See Wenxin Jiang, Arav Tewari, et al, Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters, Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1466–1478,Vien 2024

Is the ONNX specification clear and complete?

Laconic and lacunar documentation

Sometimes laconic: *what is a convolution?*

Conv - 22 ↑ Back to top

Summary

The convolution operator consumes an input tensor and a filter, and computes the output.

(Excerpt of ONNX doc.)

1 No specification of the operation

Ceil - 13

Summary

Ceil takes one input data (Tensor) and produces one output data (Tensor) where the ceil is, y = ceil(x), is applied to the tensor elementwise. If x is integral, +0, -0, NaN, or infinite, x itself is returned.

2 More or less Reflexive definition

Sometimes lacunar: *What is the value used for padding in a convolution?*

- Uhhh… zero?

Is the ONNX specification clear and complete?

Handling of special values

- Clip operator

Interpretation 1:

Clip(-0,+0,1) = min(1,max(-0,+0)) = min(1,+0) = +0

Interpretation 2:

Clip(-0,+0,1) = if -0 < +0 → +0 else if -0 > 1 → 1 else → -0 = -0

Clip(A,L,M):

For all element a of tensor A:

$$Clip(a, L, M) = \min(M, \max(a, L))$$

1 Not true for IEEE's $+0$ and -0

$$A = [\ -\infty \quad 0.0 \quad \infty \quad NaN\ ]$$

$L = NaN$ $M = NaN$

$$Y = [\ -\infty \quad 0.0 \quad \infty \quad NaN\ ]$$

2 Why not NaNs?

Is the ONNX specification clear and complete?

Handling of special values

- Max operator

Summary

Element-wise max of each of the input tensors (with Numpy-style broadcasting support). All inputs and outputs must have the same data type. This operator supports **multidirectional (i.e., Numpy-style) broadcasting**; for more details please check Broadcasting in ONNX.

Max(NaN,X) = ?

IEEE754 **`maximum(x,y)`**

is x if x>y, y if y>x, **and a quiet NaN if either operand is a NaN** [...]

IEEE754 **`maximumNumber(x,y)`**

is x if x>y, y if y>x, **and the number if one operand is a number and the other is a NaN** [...]

Which interpretation is correct?

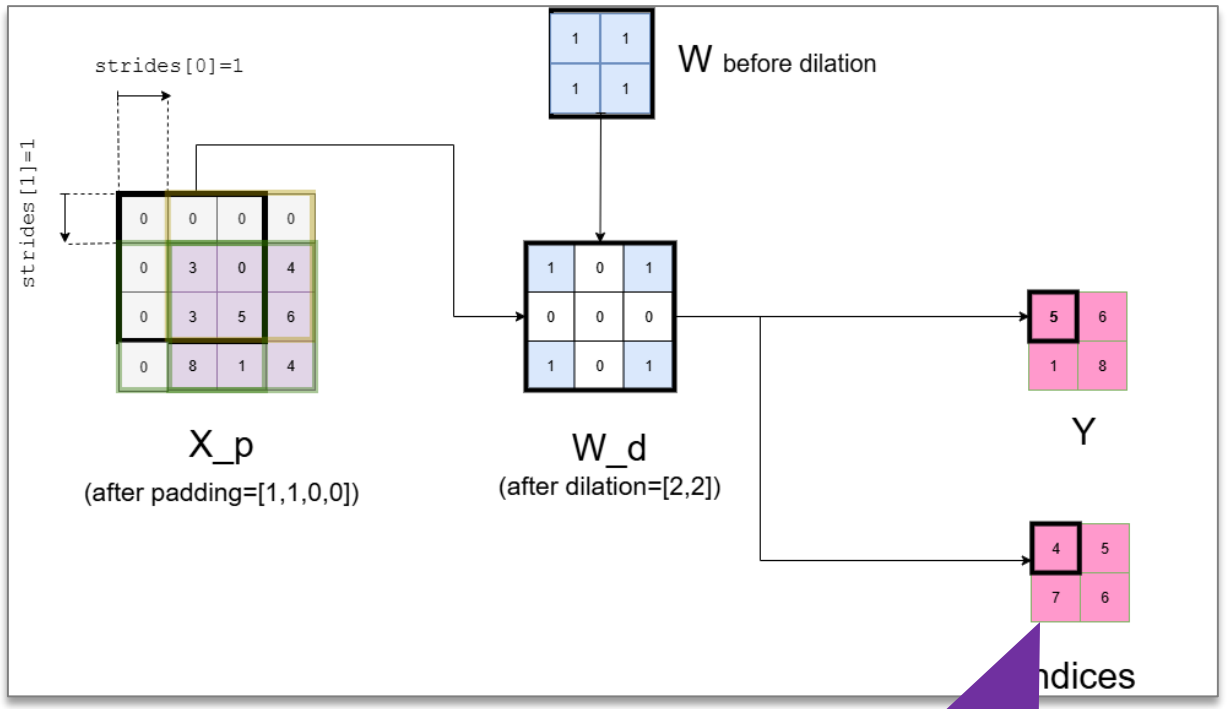
Is the ONNX specification clear and complete?

Handling of special values

- MaxPool operator

$$S, Ind = \text{MaxPool}(E)$$

- Data type: double
- Shape of E = [1, 1, 3, 3]
- kernel_shape = [2,2]
- pads = [0,0,0,0]
- dilation = [1,1]
- strides = [1,1]
- Shape of Y = [1, 1, 2, 2]
- Shape of Ind = [1, 1, 2, 2]



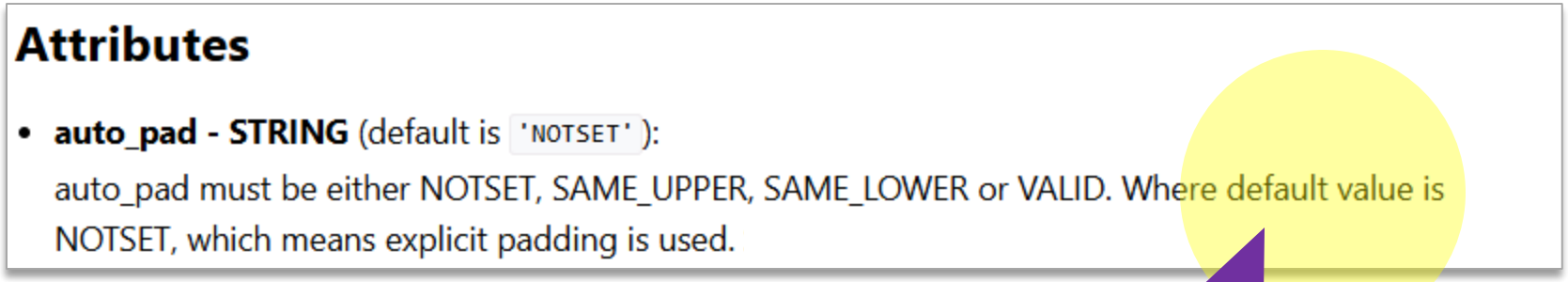
1 What happen when multiple values are equal to the max?

2 Why not –inf?

3 Index = -4 => runtime error

Do we have some specific constraints?
Default values

- Conv operator



Conv operator

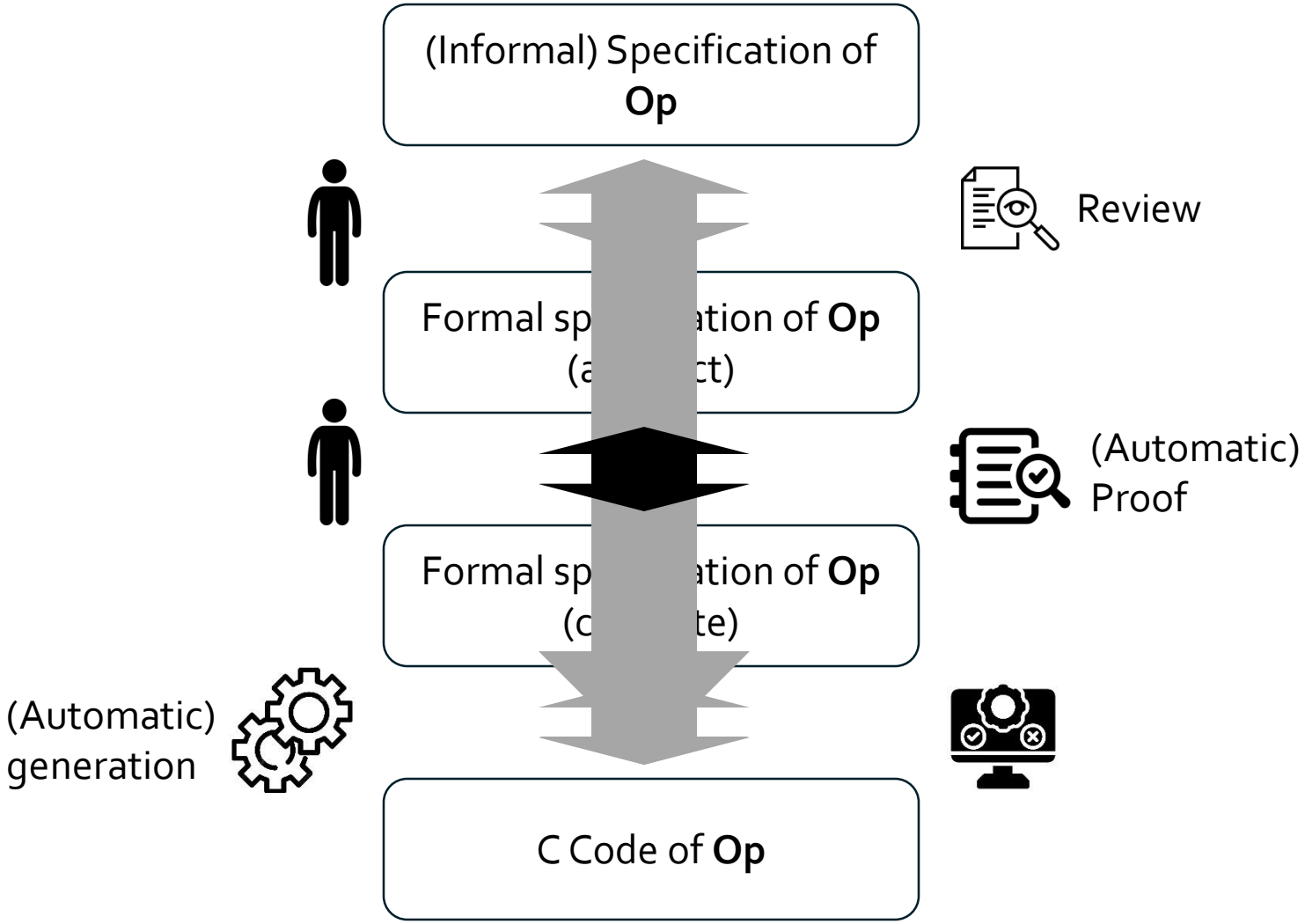
Do we accept default values?

SONNX

How.

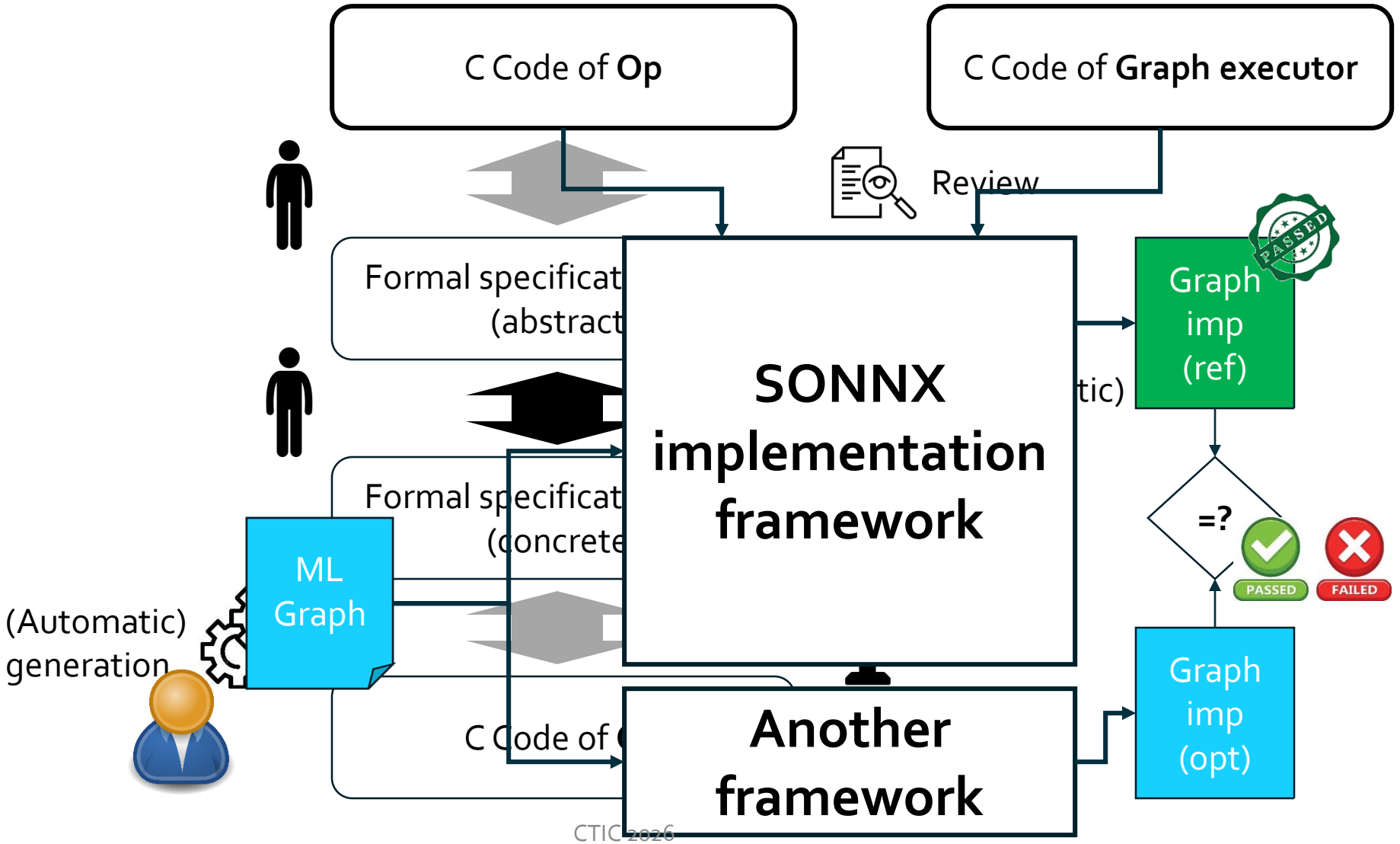
The SONNX workflow

From the specification to the implementation



The SONNX workflow

From the specification to the implementation





What.

The SONNX working group

Deliverables

(D1.a) Safety-related Profile **Scope** Definition (2024/11/01)

(D1.c) **Consolidated needs** for all industrial domains (2025/01/01)

(D2.a) ONNX safety-related Profile **requirements** (2025/02/01)

(D3.a) ONNX Safety-related profile – graph

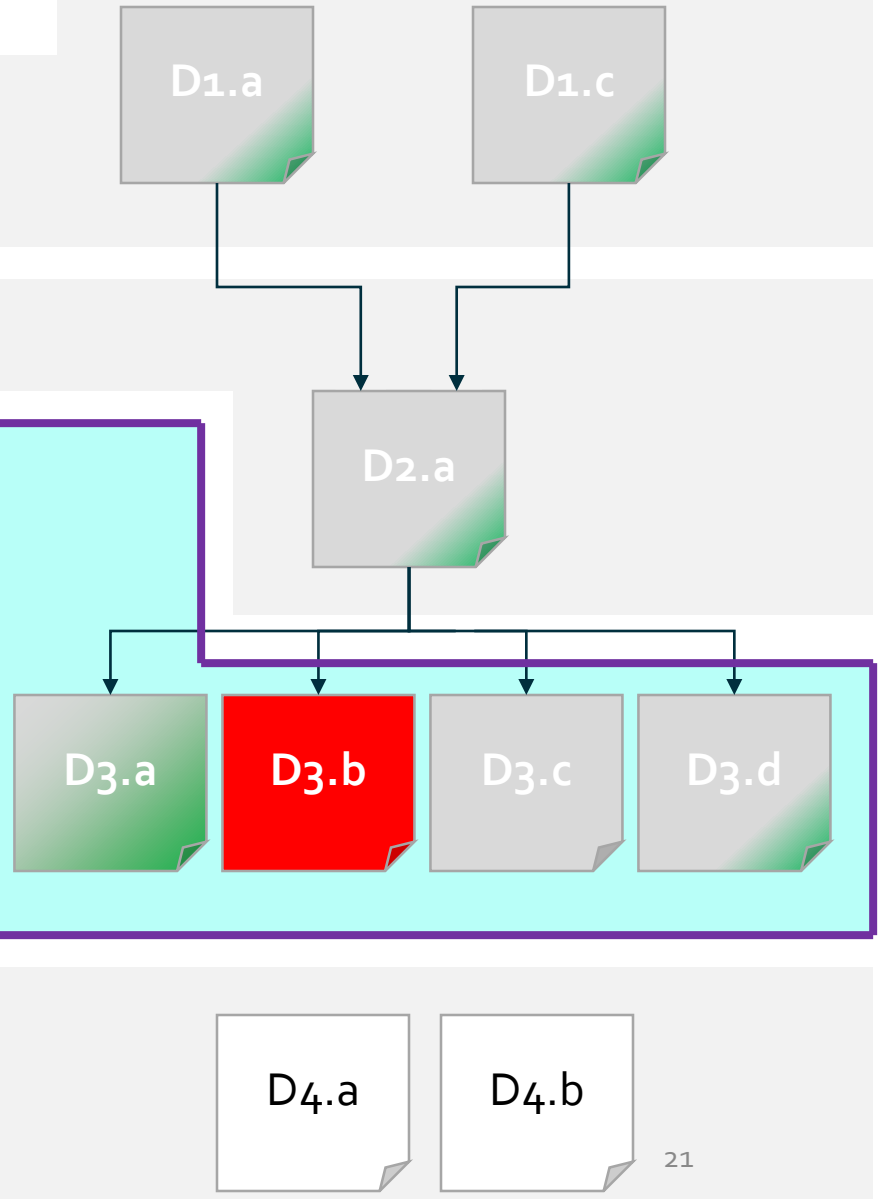
(D3.b) ONNX Safety-related profile – operators

(D3.c) ONNX Safety-related profile – format

(D3.d) ONNX Safety-related profile reference implementation

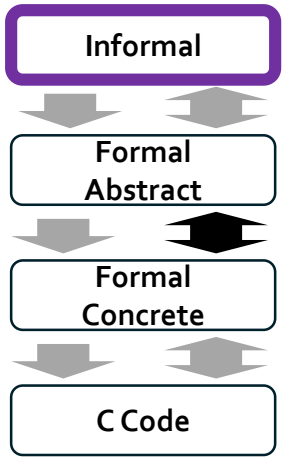
(D4.a) ONNX Safety-related profile **verification** report

(D4.b) ONNX Safety-related profile **validation** report



Informal specification

The **conv** operator - restrictions



`conv` operator

Contents

- `Convolution` operator for type real.

`Conv (real)`

Signature

`Y = conv(X,W,[B])` where

- `X` : input tensor
- `W` : convolution kernel
- `B` : optional bias
- `Y` : output tensor

Restrictions

The following restrictions apply to the `conv` operator for the SONNX profile:

Restriction	Statement	Origin
`R1`	Input tensor `X` has 2 spatial axes	Transient
`R2`	Attribute `auto_pad` is set to `NOTSET`	No default values
`R3`	Attribute `group` is set to 1 (standard convolution) or to the number of channels of the input tensor `X` (depthwise convolution)	Transient

1 Simplification of the WG's work

2 Development assurance

Informal specification

The **conv** operator – functional specification

Informal
Formal Abstract
Formal Concrete
C Code

Informal specification

Operator `conv` computes the convolution of the input tensor `X` with the kernel `W` and adds bias `B` to the result. Two types of convolutions are supported

Standard conv

A *standard co*
For a given o
corresponds

$$Y[b,c,m,n] = \sum_{i=0}^{dW_1-1} \sum_{j=0}^{dW_2-1} \sum_{z=0}^{dW_3-1} (X_p[b,i,m \cdot \text{strides}[0]+j, n \cdot \text{strides}[1]+z] \cdot W_d[c,i,j,z]) + B_b[c]$$

Simple, "naïve" formulation

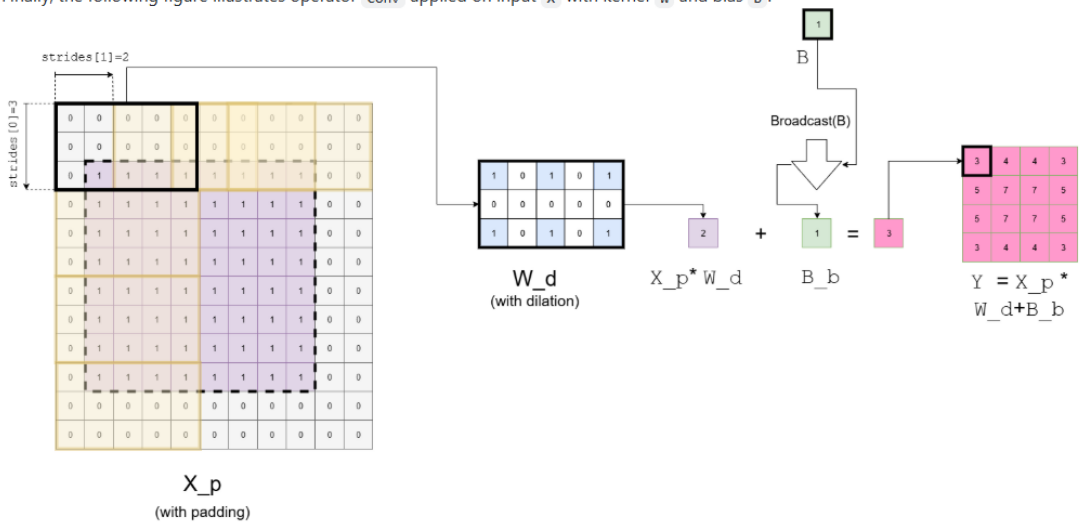
The mathematical definition of the operator is given hereafter:

$$Y[b,c,m,n] = \sum_{i=0}^{dW_1-1} \sum_{j=0}^{dW_2-1} \sum_{z=0}^{dW_3-1} (X_p[b,i,m \cdot \text{strides}[0]+j, n \cdot \text{strides}[1]+z] \cdot W_d[c,i,j,z]) + B_b[c]$$

Where

- $b \in [0, dY_0 - 1]$ is the batch index. dY_0 is the batch size of output `Y`
- $c \in [0, dY_1 - 1]$ is the data channel. dY_1 is the number of data channels of output `Y`
- $m \in [0, dY_2 - 1]$ is the index of the first spatial axis of output `Y`
- $n \in [0, dY_3 - 1]$ is the index of the second spatial axis of output `Y`
- dW_1 is the number of feature maps of kernel `W`
- dW_2 is the size of the first spatial axis of kernel `W`
- dW_3 is the size of the second spatial axis of kernel `W`
- `strides` is an attribute of the operator. It will be described later in this section.
- $X_p = \text{pad}(X, pads)$ is the padded version of the input tensor `X`. Function pad applies zero
  ONNX `Pad` operator).
- $W_d = \text{dilation}(W, dilations)$ is the dilated version of the kernel `W`. Function dilation expa
  elements as specified by the dilations attribute. Its definition is given later.
- $B_b = \text{broadcast}(B, (dY_0, dY_1, dY_2, dY_3))$ is the broadcasted version of bias `B`. Function br
  dimensions and batch dimension of the output `Y`. It takes as argument the bias `B` and t

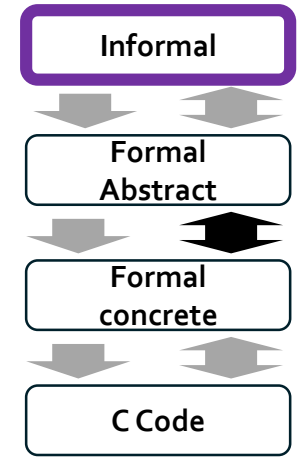
Finally, the following figure illustrates operator `Conv` applied on input `X` with kernel `W` and bias `B`:



Illustrations

Informal specification

The **conv** operator – functional specification – constraints



`X`: tensor of real

Tensor `X` is the input tensor on which convolution with kernel `W` is computed.

The shape of tensor `X` is (dX_0, dX_1, dX_2, dX_3), where

- dX_0 is the batch size of input `X`.
- dX_1 is the number of data channels of input `X`.
- dX_2 and dX_3 are the sizes of the input for the two spatial axes (height and width).

Constraints

- `C1`: Number of spatial axes of tensor `X`
  - Statement: The number of spatial axes of tensor `X` is 2. `R1`
  - Rationale: This restriction is introduced to reduce the specification effort. It matches the industrial use cases
- `C2`
- `C3`
- `C4`

- `C3`: Consistency between the shape of tensors `X`, `W`, `Y` and attributes `pads`, `dilations` and `strides`
  - Statement:
    - $\left\lfloor \frac{alpha - ((dilations[0] \cdot dW_2 - 1) + 1)}{strides[0]} \right\rfloor + 1 = dY_2$ with $alpha = dX_2 + pads[0] + pads[2]$

    and

    - $\left\lfloor \frac{beta - ((dilations[1] \cdot dW_3 - 1) + 1)}{strides[1]} \right\rfloor + 1 = dY_3$ with $beta = dX_3 + pads[1] + pads[3]$

Preconditions relating arguments and attributes

Informal specification
Traceability

1 Specification item

Function

[E_DIV_REAL_FUNC_010]

Operator **Div** divides input tensors A and B element-wise and stores the
in output tensor C. If i is a tensor index, each element $C[i]$ is the result o
dividing $A[i]$ by $B[i]$.

The mathematical definition of the operator is given hereafter.

For any tensor index i:

$$C[i] = \begin{cases} A[i]/B[i] & \text{if } B[i] \text{ is different from } 0 \\ \text{undefined} & \text{otherwise} \end{cases}$$

[END]

The effect of the operator is illustrated on the following examples.

Example 1

$$A = \begin{bmatrix} 6.1 & 9.5 & 35.7 \end{bmatrix}$$

$$B = \begin{bmatrix} 3 & 3.3 & 5.1 \end{bmatrix}$$

$$C = \frac{A}{B} = \begin{bmatrix} 6.1/3 & 9.5/3.3 & 35.7/5.1 \end{bmatrix}$$

2 Specification item

Inputs

A: real tensor

Numerator of the division.

Constraints

- `[E_DIV_REAL_CONSTR_A_010]` Shape consistency
  - Statement: Tensors A, B, and C shall have the same shape.

B: real tensor

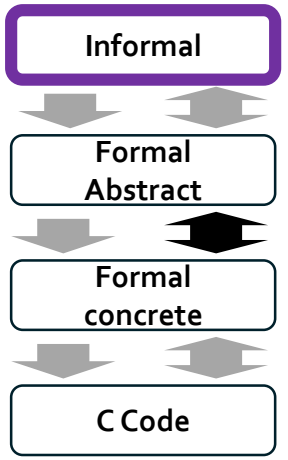
Denominator of the division.

Constraints

- `[E_DIV_REAL_CONSTR_B_010]` Shape consistency
  - Statement: see constraint `E_DIV_REAL_CONSTR_A_010` on tensor A.
- `[E_DIV_REAL_CONSTR_B_020]` Avoid undefined behaviour
  - Statement: $\forall i, B[i] \neq 0$

Informal specification

Error conditions *vs.* total functions



Div (int, int)

Operator **Div** divides input tensors A and B element-wise and stores the result in output tensor C.

The result of the division is the algebraic quotient of $A[i]$ by $B[i]$ with any fractional part discarded.

Error conditions

The behaviour in case of a null denominator is implementation dependent.

1 Error condition

Add (int, int)

For unsigned values (type `uintn`):

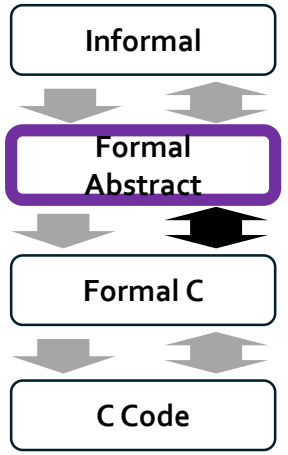
$$Y[i] = \begin{cases} A[i] + B[i] - k.2^n & \text{if } A[i] + B[i] > 2^n - 1 \\ A[i] + B[i] & \text{otherwise} \end{cases}$$

with $k \in N$ such that $0 \leq A[i] + B[i] - k.2^n < 2^n$

2 Total function (actual semantics)

Formal specification

The **conv** operator – Abstract **tensors**




```
type data 'a = map (list int) 'a
```

```
type tensor 'a = {
  dims : list int ;
  data : data 'a ;
  background : 'a ; (* default value, or value for 0-dimensions tensor *)
}
```

A tensor is a map from an index to a value

```
(** Formalization of Tensor *)
module Tensor
  use int.Int
  use map.Map
  use list.List
  use Range

  type data 'a = map (list int) 'a

  type tensor 'a = {
    dims : list int ;
    data : data 'a ;
    background : 'a ; (* default value, or value for 0-dimensions tensor *)
  }

  (** Constant Tensor *)
  let ghost function const (v : 'a) (bg : 'a) (ds : list int): tensor 'a
    ensures { result.dims = ds }
    requires { positive ds }
    ensures { result.background = bg }
    ensures { forall k. valid k ds -> result k = v }
    (*proof*)
    = { dims = ds ; data = pure { fun k -> if valid k ds then v else bg } ; background = bg }
    (*qed*)

  (** Constant & Null *)
  goal zero_is_const: forall e : 'a, ds. positive ds -> zero e ds == const e e ds

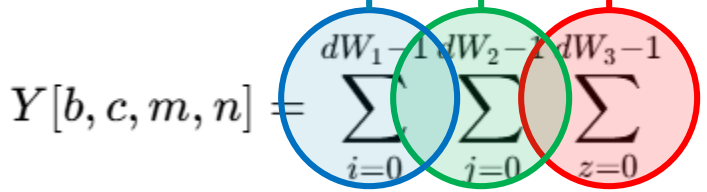
end
```

Why3 platform Inria

Formal specification

The **conv** operator – Abstract specification

1 Traceability needs to be improved...




```
let rec ghost function w_channels_calculate (x w :tensor real) (c:int )(cc:int)
(i:int) (j:int) (n m y_h y_w: int)  (str_h str_w :int) (dil_h dil_w :int) (pad_top
pad_left :int) : real
 variant { c - cc }
 =   if cc >= c then   0.0
  else  w_lines_calculate x w cc i 0 j n m y_h y_w str_h str_w dil_h dil_w pad_top pad_left +
w_channels_calculate x w c Int.(cc + 1) i j n m y_h y_w str_h str_w dil_h dil_w pad_top
pad_left
```

```
let rec ghost function w_lines_calculate (x w : tensor real) (c: int) (i:int) (ii:int)
(j:int) (n m y_h y_w: int)  (str_h str_w :int) (dil_h dil_w :int) (pad_top pad_left
:int) : real
 variant { i - ii }
 =   if ii >= i then      0.0
  else  w_cools_calculate x w c ii j 0 n m y_h y_w str_h str_w dil_h dil_w pad_top pad_left +
w_lines_calculate x w c i Int.(ii + 1) j n m y_h y_w str_h str_w dil_h dil_w pad_top pad_left
```

```
let rec ghost function w_cools_calculate (x w : tensor real) (c: int) (i:int) (j:int)
(jj:int) (n m y_h y_w: int)  (str_h str_w :int) (dil_h dil_w :int) (pad_top pad_left
:int) : real
 variant { j - jj }
 =   if jj >= j then      0.0
  else      let x_h = y_h * str_h + i * dil_h in
    let x_w = y_w * str_w + jj * dil_w in
    let x_val = coords_from_X_p x (Cons n (Cons c (Cons x_h (Cons x_w Nil))) )
pad_top pad_left in
    let w_val = w.data (Cons m (Cons c (Cons i (Cons jj Nil)))) in
    Real.(x_val * w_val + w_cools_calculate x w c i j Int.(jj + 1) n m y_h y_w str_h
str_w dil_h dil_w pad_top pad_left)
```

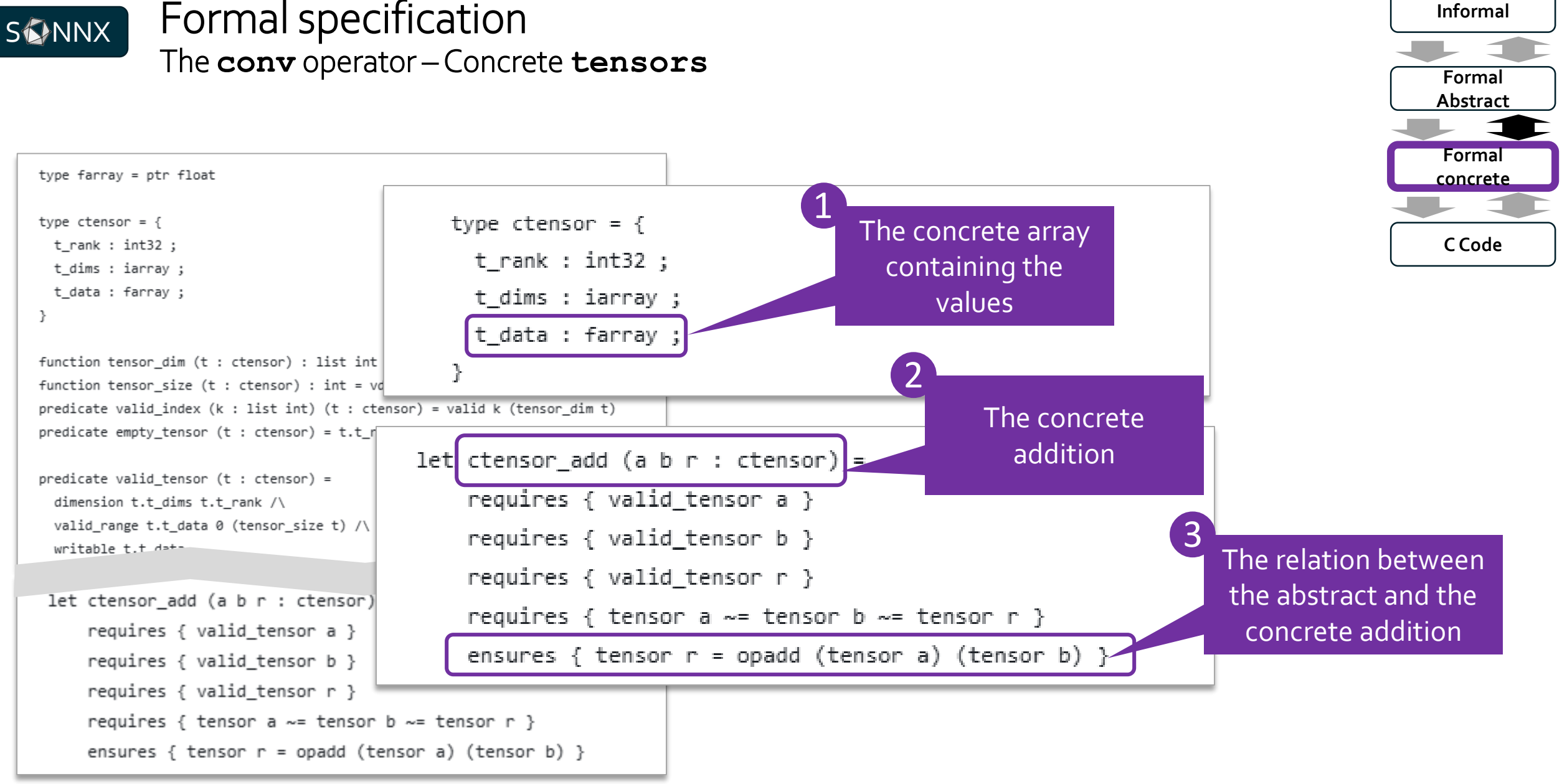
2 **Abstract properties needs to be added and proved**

- Conv(X1 + X2, W) = Conv(X1, W) + Conv(X2, W)
- Conv(a X, W) = a Conv(X, W)
- Conv(X, a W) = a Conv(X, W)
- Conv(X, W1 + W2) = Conv(X, W1) + Conv(X
- Conv(0, W) = 0
- Conv(X, 0) = 0
- Etc.

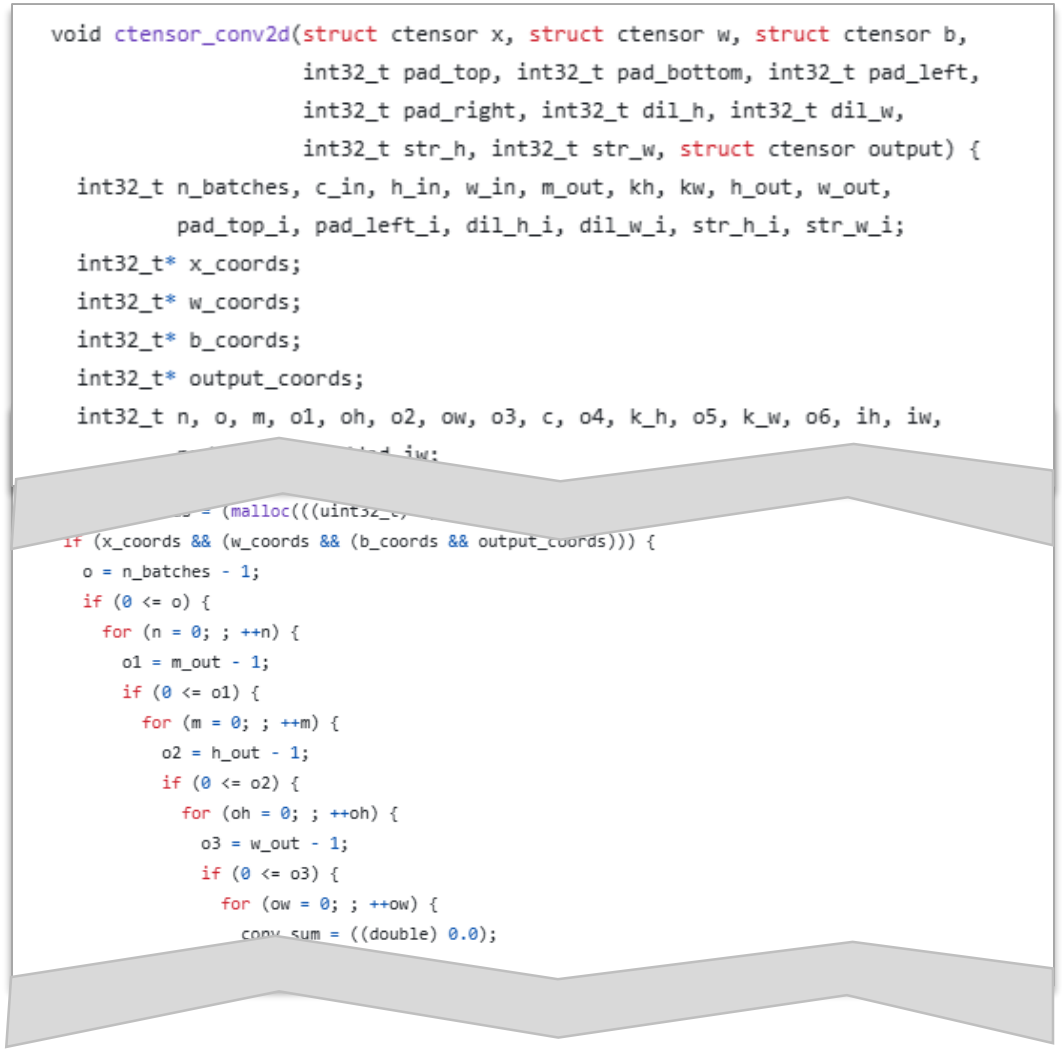
See AIDGE

Formal specification

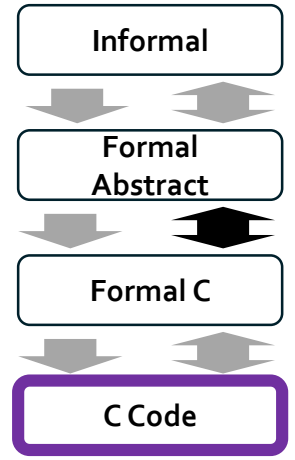
The **conv** operator – Concrete **tensors**



Code generation



```
void ctensor_conv2d(struct ctensor x, struct ctensor w, struct ctensor b,
                    int32_t pad_top, int32_t pad_bottom, int32_t pad_left,
                    int32_t pad_right, int32_t dil_h, int32_t dil_w,
                    int32_t str_h, int32_t str_w, struct ctensor output) {
  int32_t n_batches, c_in, h_in, w_in, m_out, kh, kw, h_out, w_out,
          pad_top_i, pad_left_i, dil_h_i, dil_w_i, str_h_i, str_w_i;
  int32_t* x_coords;
  int32_t* w_coords;
  int32_t* b_coords;
  int32_t* output_coords;
  int32_t n, o, m, o1, oh, o2, ow, o3, c, o4, k_h, o5, k_w, o6, ih, iw,
                                                              iw;
                                                    (malloc(((uint32_t
 if (x_coords && (w_coords && (b_coords && output_coords))) {
   o = n_batches - 1;
   if (0 <= o) {
     for (n = 0; ; ++n) {
       o1 = m_out - 1;
       if (0 <= o1) {
         for (m = 0; ; ++m) {
           o2 = h_out - 1;
           if (0 <= o2) {
             for (oh = 0; ; ++oh) {
               o3 = w_out - 1;
               if (0 <= o3) {
                 for (ow = 0; ; ++ow) {
                   conv_sum = ((double) 0.0);
```



- Naïve implementation
  - Simple, traceable to the specification
  - but very slow…
- Tests shall be conducted on small tensors / kernels

Testing

- Testing
  - Comprehensive functional tests (to be) provided for each operator
  - Based on explicit **equivalence classes** and **operator properties**
  - (Also contributes to increasing our confidence on the informal specification)

- Numerical accuracy (work of Franck VEDRINE, CEA)

Numerical Accuracy (2/3)

Introduced error

- Introduced error
  - Scalar multiplication
  - Matrix multiplication

Introduced error

$$|IE(f)| \leq \max(|a|,|b|) \times \max(|c|,|d|) \times \mathbf{u}$$

Naïve algorithm

```
// A matrix of dimension n x p, B matrix of
dimension p x q.
// C result matrix of dimension n x q
for (int i = 0; i < n; ++i)
  for (int j = 0; j < q; ++j) {
    C[i][j] = 0;
    for (int k = 0; k < p; ++k)
      C[i][j] += A[i][k]*B[k][j];
```

```
// A matrix of dimension n x p, B matrix of
dimension p x q.
// C result matrix of dimension n x q
for (int i = 0; i < n; ++i)
  for (int j = 0; j < q; ++j) {
    C[i][j] = 0;
    // IE(C[i][j]) = 0
      C[i][j] += A[i][0]*B[0][j];
    // the absolute value of every coefficient of A
is bound by 'a'
    // the absolute value of every coefficient of B
is bound by 'b'
    // |IE(C[i][j])| <= a*b*u
    // |C[i][j]| <= a*b*(1+u)
      C[i][j] += A[i][1]*B[1][j];
    // |IE(C[i][j])| <= a*b*u + a*b*u + 2*a*b*
(1+u)*u = a*b*u*(4+2u)
    // |C[i][j])| <= (a*b*(1+u) + a*b*(1+u))*(1+u) =
2*a*b*(1+u)²
      C[i][j] += A[i][2]*B[2][j];
    // |IE(C[i][j])| <= a*b*u*(4+2u) + a*b*u +
(2*a*b*(1+u)² + a*b*(1+u))*u = a*b*u*(8+7u+2u²)
    // |C[i][j]| <= (2*a*b*(1+u)² + a*b*(1+u))*(1+u)
= (3+2*u)*a*b*(1+u)²
    for (int k = 3; k < p; ++k)
      C[i][j] += A[i][k]*B[k][j];
```

Propagated error

$$|PE(f)| \leq |y_{val}| \times |x_{err}| + |x_{val}| \times |y_{err}| + |\mathcal{O}(X_{err}^2)|$$

$$|IE(C[i][j])| \leq$$

$$\left(\frac{(1+u)^{p+1}-1}{u} \times (1+u)^2 - p - 1\right) \times a \times b$$

$$u = \frac{\varepsilon}{2}$$

Numerical Accuracy (3/3)

Automatic computation

- For the division

```
Tensor<SymbolicDomainError> A, B;

/* A, B symbolic initialization */

auto result = [&A,&B](auto I) {
  return (B[I].real != 0) ? A[I] / B[I] :
      /* undefined */ SymbolicDomainError::undef();
};

for (auto I : A.indexes()) {
   auto a = A[I];
   auto b = B[I];
   if (b.real != 0 && b.real + b.err != 0) {
      auto c = result(I);
      double bound = std::abs(a.err / b.real) +
        std::abs(a.real * b.err / (b.real * b.real));
      assert(std::abs(c.err) <= bound + 1e-12);
   }
}
```

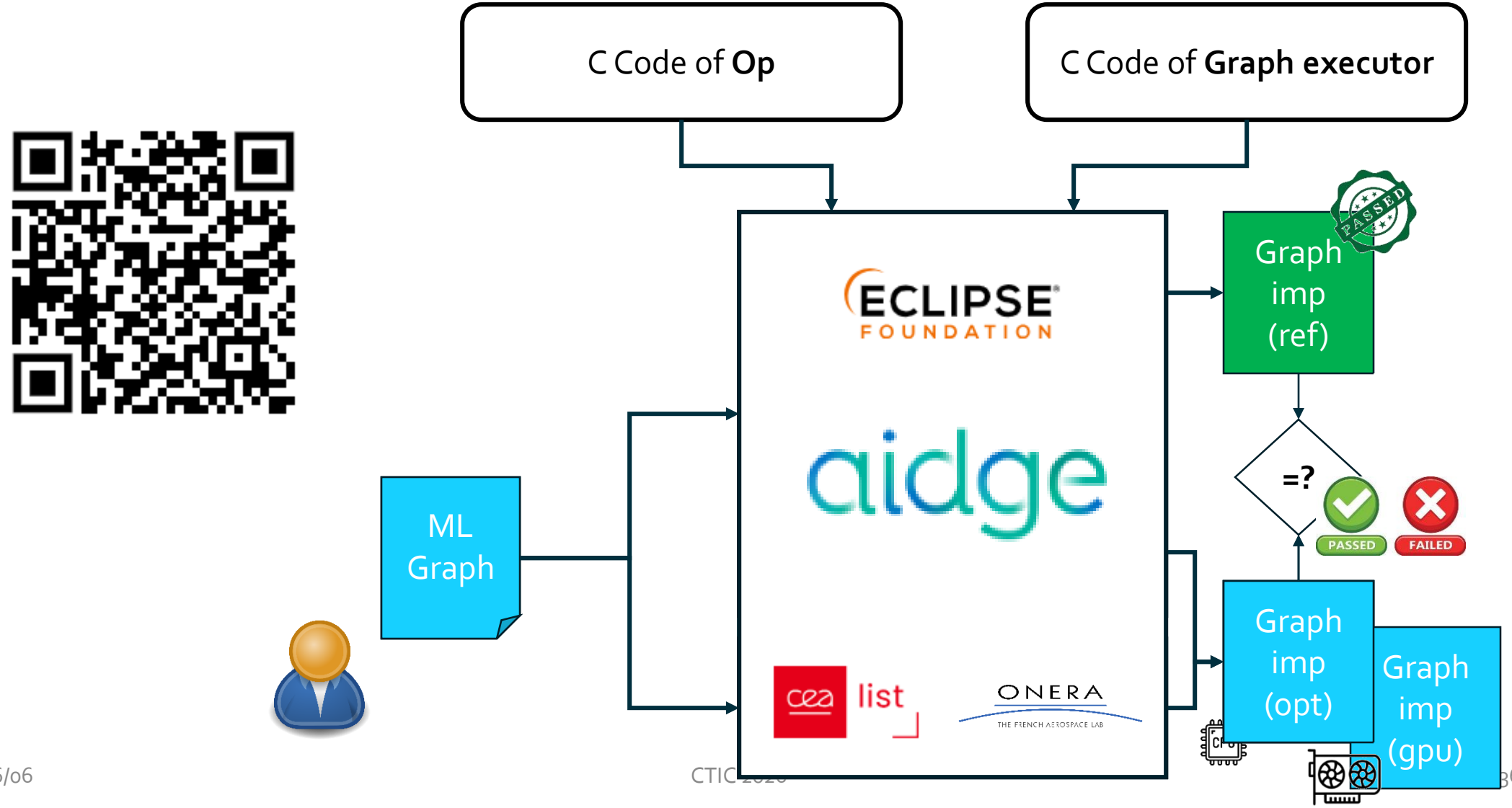
$$|IE(C[i])| \leq \frac{|A[I]|}{|B[I]|} \times \mathbf{u}.$$

$$|PE(g)| \leq \frac{1}{|y_{val}|} \times |x_{err}| + \frac{|x_{val}|}{y_{val}^2} \times |y_{err}| + |\mathcal{O}(X_{err}^2)|$$

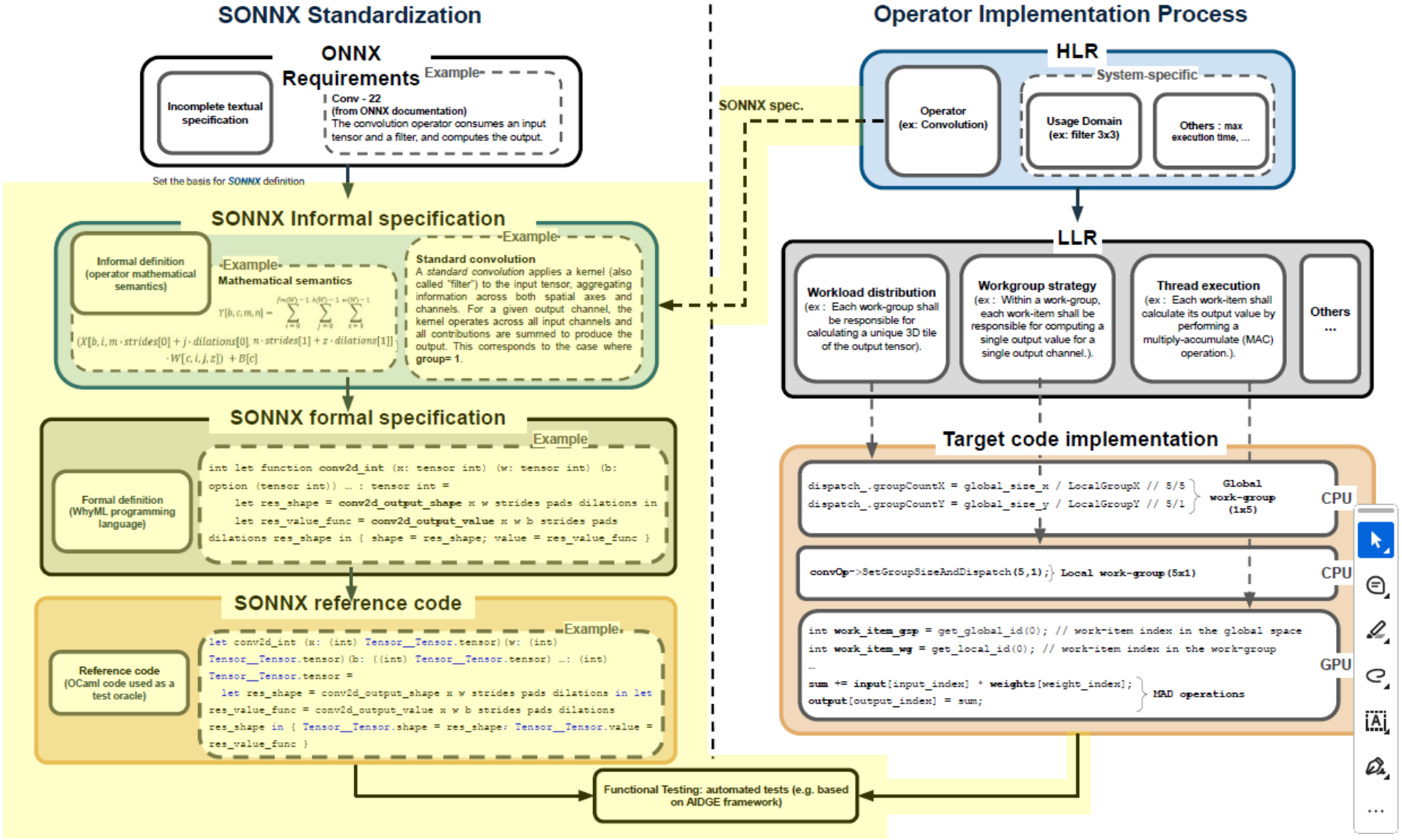
IE + PE = total error

The SONNX workflow

From the specification to the implementation



An industrial application of SONNX…



Conclusion
Where are we? What's next?

- Where are we…
  - Operators
    - First set of
      - informal spec
      - formal spec
      - Tests
      - Reference implementation
  - Graph
    - Formal spec on-going
  - Integration in Aidge on-going

Main contributors…

- Henri BELFY (Thales AVS)
- Loïc CORRENSON (CEA)
- João COSTA MACHADO (Critical Software)
- Jean-Loup FARGES (ONERA)
- **Eric JENN (IRT Saint-Exupery) – Co-lead**
- Edoardo MANINO (Manchester University)
- Ricardo MOREIRA DA SILVA (Critical Software)
- **Jean SOUYRIS (Airbus) – Co-lead**
- Mariem TURKI (IRT Saint Exupery)
- Franck VEDRINE (CEA)

SONNX github repo

We Need You!



With support from ANITI **A**rtificial and **N**atural **I**ntelligence **T**oulouse **I**nstitute